Workshop 3: Data Model and Validation in Google Sheets

Legal History Hub · Max Planck Institute for Legal History and Legal Theory

Mag. Christian Steiner MA · Digital Humanities Craft OG · 2026

These slides were authored with Claude Code in VS Code and rendered to HTML with Marp. The source is a Markdown file in the repo, versioned alongside the rest of the project. The workflow is itself part of what you will learn.

Agenda

1. **Wide, Long, Tidy** – why our Sheet looks the way it does (45 min)
2. **The data model with Claude Code** – read, adapt, validate (45 min)

Break (10–15 min)

3. **Google Sheets as a CMS** – Tables, dropdowns, views (45 min)
4. **Skills and plugins** – extending Claude Code (30 min)
5. **Wrap-up and outlook** (15 min)

Terms you will hear today (1/2)

Term	Short
FAIR	Findable, Accessible, Interoperable, Reusable (data quality principle)
PID	Persistent Identifier, e.g. ORCID, GND, ROR, Wikidata Q-number
Authority file	central list of names with their IDs
Foreign Key	reference into another table (here: `project_id`)
Junction Table	table that resolves a Many-to-Many relationship
Many-to-Many	one project has many people, one person has many projects

Terms you will hear today (2/2)

Term	Short
Singleton	field with exactly one value per entity (e.g. title)
Source of Truth	the one place where the correct value lives
Wide / Long / Tidy	data formats (covered in detail shortly)
1NF	First Normal Form: each cell holds exactly one value
Enum	closed list of values (e.g. status: `active`, `completed`, `planned`)
Spill Range	formula that automatically fills several result rows

These two slides stay as a reference. Flip back here whenever you need a definition.

Block 1: Wide, Long, Tidy

The pilot experiment: the first attempt

Between WS2 and WS3, **Polina built her own data model** as an experiment. → Polina's pilot Sheet

6 tabs, relationally normalised: separate tabs for projects, people, organisations, subjects, keywords, and regions.

Conceptually clean: local IDs (`pers-003` , `subj-001`), ORCIDs, GND numbers. That is **relational thinking** and fundamentally right.

What the pilot model got right

	A	B	C	D	E	F	G	H	I	J	K
1	**person_id**	**name**	**orcid_id**	**orcid_status**							
2	pers-001	Wagner, Andreas	0000-0003-1835-1653	confir…							
3	pers-002	Birr, Christiane		none							
4	pers-003	Duve, Thomas	0000-0002-1658-4173	confir…							
5	pers-004	Lutz-Bachmann, Matth	0009-0006-4006-8773	confir…							
6	pers-005	König, Florian	0009-0006-2438-0513	confir…							
7	pers-006	Rico Carmona, Cindy	0000-0001-5095-1793	confir…							
8	pers-007	Solonets, Polina	0000-0001-5223-1220	confir…							
9	pers-008	Hugel, Marie-Astrid		none							
10	pers-009	Schweighöfer, Stefan	0009-0001-6421-736X	confir…							
11	pers-010	Mejía, Pilar	0000-0002-2690-1734	confir…							
12	pers-011	Soler, Ana Isabel		none							
13	pers-012	Aguiar, Nádia dos Santos		none							
14	pers-013	Goncalves, Larissa		none							
15	pers-014	Collin, Peter		none							
16	pers-015	Wolf, Johanna	0000-0001-6364-8902	confir…							
17	pers-016	Ebbertz, Matthias	0000-0001-6440-2453	confir…							
18	pers-017	Vesper, Tim-Niklas		none							
19	pers-018	Michel, Lisa		none							
20	pers-019	Gödde, Ben		none							
21	pers-020	Wolf, Paulina		none							
22	pers-021	Damler, Daniel		none							
23	pers-022	Egío García, José Luis	0000-0002-9256-8490	confir…							
24	pers-023	Schüssler, Rudolf		none							
25											
26											
27											
28											

Where it breaks: separator lists

		F	G	H	I	J	K	L
1		description_sh	description_sh	description_sh	description_de	description_en	description_es	creator
2	e S	Una edición digit	A digital edition c	Eine digitale Aus	Die digitale Quel	The Digital Collection of Sources contain	La Colección Digital de Fuentes contiene textos completos y copias digitale	pers-003|PI;pers
3	nón	Eine digitale Que	A digital source e	Una edición digit	In der Quellensa	In the source collection, sets of rules are	En la colección de fuentes se editan conjuntos de normas que muestran la	pers-014|PI;pers
4	listó	Ein historisches '	A historical dictic	Un diccionario hi	Historisches Wöı	The Historical Dictionary of Canon Law ir	Diccionario Histórico de Derecho Canónico en Hispanoamérica y Filipinas (	pers-003|PI;pers
5	nán	Gonzalo Fernán	Gonzalo Fernán	Gonzalo Fernán	Er war das Idol v	He was the idol of Alexander von Humbo	Fue el ídolo de Alexander von Humboldt, quien lo admiraba como el «Plinio	pers-021|PI
6	iad	Die Dissertation	This dissertation	La tesis doctoral	Die Dissertation	This dissertation examines the regulatior	Esta tesis examina la regulación de las relaciones laborales y las condicion	pers-016|PI
7								
8								
9								
10								
11								
12								
13								
14								
15								
16								
17								
18								
19								
20								
21								
22								
23								
24								
25								
26								
27								
28								

Why is this a problem?

Database theory has a rule called the **First Normal Form (1NF)**:

> *"An entry in a table is not decomposable."*
> – Edgar F. Codd, *A Relational Model of Data for Large Shared Data Banks*, 1970.

Codd was an IBM researcher and the inventor of relational databases. His rule says: **a cell holds exactly one value.** No lists, no separators.

	J	K	L	M	N
1	**on_en**	**description_es**	**creator**	**subject**	**keywords_de**
2	al Collection of Sources contain	La Colección Digital de Fuentes contiene textos completos y copias digitale	pers-003|PI;pers	subj-001;subj-002;subj-003	kooperation;digitale-geisteswissenschaften;quelle
3	urce collection, sets of rules are	En la colección de fuentes se editan conjuntos de normas que muestran la	pers-014|PI;pers	subj-004	digital-humanities;sonderordnungen
4	rical Dictionary of Canon Law ir	Diccionario Histórico de Derecho Canónico en Hispanoamérica y Filipinas (	pers-003|PI;pers	subj-001	iberische-welten;normativität;religion
5	he idol of Alexander von Humbo	Fue el ídolo de Alexander von Humboldt, quien lo admiraba como el «Plinio	pers-021|PI	subj-005	transmedia-historytelling;rechtshistorische-praxis
6	ertation examines the regulation	Esta tesis examina la regulación de las relaciones laborales y las condicion	pers-016|PI	subj-004	sonderordnungen
7					
8					
9					
10					
11					
12					
13					

Why many columns don't help either

project_id	person_1	role_1	person_2	role_2	...	person_15
proj-001	Duve	PI	L-B	PI	...	

How many column pairs are enough? With 3 people, 12 columns sit empty. With 20, you would have to change the table structure.

A table full of empty cells, endless horizontal scrolling, a hard upper limit. In DB theory: **Repeating Groups** (Codd 1970).

Three formats compared

Format	What it looks like	Problem
Wide + separator lists	`creator = "Duve, T.; L-B, M."` in one cell	Sheets sees only text, no filtering/counting
Relational normalised	dedicated people table, only IDs in projects	Sheets has no joins, humans have to merge mentally
Hybrid (wide + long)	wide for singletons, long tab for relationships	redundancy in title columns, but Sheets-friendly

The pilot model was format 2 (plus separator lists). Today we build format 3.

Why not format 2? Because Sheets is a user interface, not a database. Humans read, click, filter. Joins do not exist there.

The solution: a dedicated tab in long format

project_id	person	role
proj-001	Duve, Thomas	PI
proj-001	Lutz-Bachmann, M.	PI
proj-001	König, Florian	researcher
proj-002	Collin, Peter	PI

One row per person-project-role. This is called the **long format**.

> *"Each variable is a column, each observation is a row."* – Hadley Wickham, *Tidy Data*, 2014.

The tab itself is a **junction table**: a table that resolves a many-to-many relationship.

How this looks in our Sheet

	A	B	C	D	E	F	G	H	I
1	project_id	title_de	person	role					
2	proj-001	Die Schule von Salamanca. Eine digita	Duve, Thomas	PI					
3	proj-001	Die Schule von Salamanca. Eine digita	Lutz-Bachmann, Matthias	PI					
4	proj-001	Die Schule von Salamanca. Eine digita	König, Florian	researcher					
5	proj-001	Die Schule von Salamanca. Eine digita	Birr, Christiane	researcher					
6	proj-001	Die Schule von Salamanca. Eine digita	Rico Carmona, Cindy	researcher					
7	proj-001	Die Schule von Salamanca. Eine digita	Solonets, Polina	researcher					
8	proj-001	Die Schule von Salamanca. Eine digita	Wagner, Andreas	researcher					
9	proj-001	Die Schule von Salamanca. Eine digita	Hugel, Marie-Astrid	researcher					
10	proj-001	Die Schule von Salamanca. Eine digita	Schweighöfer, Stefan	project-coordinator					
11	proj-002	Nichtstaatliches Recht der Wirtschaft.	Collin, Peter	PI					
12	proj-002	Nichtstaatliches Recht der Wirtschaft.	Wolf, Johanna	researcher					
13	proj-002	Nichtstaatliches Recht der Wirtschaft.	Wagner, Andreas	researcher					
14	proj-002	Nichtstaatliches Recht der Wirtschaft.	Solonets, Polina	researcher					
15	proj-002	Nichtstaatliches Recht der Wirtschaft.	Ebbertz, Matthias	researcher					
16	proj-002	Nichtstaatliches Recht der Wirtschaft.	Vesper, Tim-Niklas	researcher					
17	proj-002	Nichtstaatliches Recht der Wirtschaft.	Michel, Lisa	student-assistant					
18	proj-002	Nichtstaatliches Recht der Wirtschaft.	Gödde, Ben	student-assistant					
19	proj-002	Nichtstaatliches Recht der Wirtschaft.	Wolf, Paulina	student-assistant					

Pilot model → our hybrid

Pilot experiment

- Separate people table ✓
- ORCIDs and IDs ✓
- Relationships as separator lists ✗
- 9 people in one cell ✗
- Sheets cannot filter ✗

Our hybrid model

- Separate people in the long tab ✓
- ORCIDs in `authority` ✓
- One row per relationship ✓
- Filter, sort, count ✓
- Sheets can work with it ✓

Keep the good ideas, adapt the implementation to Sheets.

The core tab: the wide format in action

	A	B	C	D	E	
1	project_id	title_de	title_en	title_es	slug	descripti
2	proj-001	Die Schule von Salamanca. Eine digitale Quellensa	The School of Salamanca. A Digital Collection of S	La Escuela de Salamanca. Colección digital de fue	salamanca-school	Die digita
3	proj-002	Nichtstaatliches Recht der Wirtschaft. Die normat	Non-state law of the economy. The normative ord	Derecho económico no estatal. El orden normativ	metall-industry	In der Qu
4	proj-003	Historisches Wörterbuch des kanonischen Rechts	Historical dictionary of canon law in hispanic Ame	Diccionario Histórico de Derecho Canónico en His	canon-dictionary	Historisc
5	proj-004	Gonzalo Fernández de Oviedo	Gonzalo Fernández de Oviedo	Gonzalo Fernández de Oviedo	gonzalo-oviedo	Er war da
6	proj-005	Verhandelte Ordnung. Arbeitsbeziehungen in der V	Negotiated Order. Labour Relations in the Weimar	Orden negociado. Las relaciones laborales en la R	verhandelte-ordnung	Die Disse
7						
8						
9						
10						
11						
12						
13						
14						
15						
16						
17						
18						
19						
20						
21						
22						
23						
24						
25						

Wide vs. long: two formats

Wide

- one row per project
- many columns side by side
- good for **singletons**: fields with exactly one value (title, status, year)
- example: our `core` tab

Long (tidy)

- one row per relationship/fact
- few columns, many rows
- good for **many-to-many**: fields with many values (people, keywords)
- example: our `people` tab

Humans read wide well. Sheets functions (filter, sort, group) work well on long. Our hybrid serves both.

The rule of thumb

If a field …	Then …
has exactly **one** value (title, status, start year)	**wide** in `core`
has 1-N values, N **small and fixed** (max. 3 URLs)	**wide** with `url1`, `url2`, `url3`
has **unbounded** values (people, keywords)	**long** in a dedicated tab

Rule of thumb: *can a project have several of these? Then long.*

Our hybrid model

Tab	Format	Content
`core`	**wide**	one project per row, all singletons
`people`	long	one person-role per row
`institutions`	long	one institutional relationship per row
`subjects`	long	one subject per row
`regions`	long	one region per row
`keywords`	long	one keyword per row

Why "hybrid"? Because we don't pick "either wide or long". We use both side by side. Each field goes into the format that fits its cardinality.

Grouping: read long as if it were wide

	A	B	C	D	E	F	G	H	I	J
1	project_id	title_de	keyword							
	project_id: proj-001 Anzahl: 6		Ausgefüllt: 100%							
2	proj-001	Die Schule von Salamanca. Eine digitale Quellensa	kooperation							
3	proj-001	Die Schule von Salamanca. Eine digitale Quellensa	digitale-geisteswissenschaften							
4	proj-001	Die Schule von Salamanca. Eine digitale Quellensa	quellen							
5	proj-001	Die Schule von Salamanca. Eine digitale Quellensa	iberische-welten							
6	proj-001	Die Schule von Salamanca. Eine digitale Quellensa	normativität							
7	proj-001	Die Schule von Salamanca. Eine digitale Quellensa	religion							
	project_id: proj-002 Anzahl: 2		Ausgefüllt: 100%							
8	proj-002	Nichtstaatliches Recht der Wirtschaft. Die normati	digitale-geisteswissenschaften							
9	proj-002	Nichtstaatliches Recht der Wirtschaft. Die normati	sonderordnungen							
	project_id: proj-003 Anzahl: 3		Ausgefüllt: 100%							
10	proj-003	Historisches Wörterbuch des kanonischen Rechts	iberische-welten							
11	proj-003	Historisches Wörterbuch des kanonischen Rechts	normativität							
12	proj-003	Historisches Wörterbuch des kanonischen Rechts	religion							
	project_id: proj-004 Anzahl: 2		Ausgefüllt: 100%							
13	proj-004	Gonzalo Fernández de Oviedo	transmedia-historytelling							

Lessons from the pilot experiment

1. **Relational thinking is valuable.** Local IDs, ORCIDs, lookup tables. We keep these.
2. **Separator lists do not fit Sheets.** The `creator` column with 9 pipe-separated entries is unusable for humans and for Sheets.
3. **Sheets is an editor, not a database.** The data model has to be written for humans. The power (joins, enrichment) emerges at build time in Claude Code.

Modeling is a conversation, not a one-shot.

Two AIs in the same Sheet: a cautionary tale

During workshop preparation, two AIs worked on this Sheet in parallel:

- **Claude** via the "Claude in Chrome" browser extension (drives a visible Chrome tab: navigates, clicks, reads cells, takes screenshots)
- **Gemini-in-Sheets** as a Google Workspace sidebar (built in)

Four iterations: three went well, one went spectacularly wrong. The one that went wrong teaches the most.

Why I am showing you this: you will work with AIs. You have to know **when you can trust them** and when not.

Iteration 4: what Gemini reported

Task: "*Insert a new column* `title_de` *with an* `XLOOKUP` *formula in each of the 5 long tabs.*"

Gemini's self-report (translated from German):

> *"I successfully inserted the new column 'Title De' into the 5 requested tabs. In the people, institutions, subjects, regions, and keywords sheets, a new column B was added in each case. The formula* `=XLOOKUP(A2, core!$A$2:$A$6, core!$B$2:$B$6, '')` *was entered. The validation rules were preserved."*

Sounds convincing. Sounds finished. Sounds correct.

What was actually in the Sheet

Only **one** tab (`people`) had been touched, and it had **eight** new columns:

```
A           B          C           D          E          F          G          H
project_id  Title De   Column 10   Title De   Title De   Column 7   Column 6   Column 5
```

All 26 data rows in the new columns: `#ERROR!` . `role` was pushed out of the viewport.

Repair: `Ctrl+Z` × 20.

Gemini's report was a **plausible description of the expected result** – classic LLM hallucination: detailed, coherent, wrong.

Three rules for working with AI

Rule	What it means in practice
Always verify.	Never trust an AI report without checking the artefact itself. In Sheets: `Ctrl+End` (jumps to the last used cell – exposes hidden extra columns/rows), filter for suspect values. In code: `git diff`, unit tests (Claude Code can write them).
Small over complex.	Five atomic tasks beat one big one. Each step is then individually verifiable. Complex multi-step tasks fail more quietly.
A report is not proof.	"*I did X*" is a claim. The proof lives in the artefact, not in the report about it.

These three rules apply to every AI, not just Gemini. Including Claude Code.

Block 2: The data model with Claude Code

How we work in Block 2

From now on: **live in Claude Code**. You prompt yourselves.

Technical setup:

- Claude Code (≥ 2.0.73) on your machines (from WS2)
- The repo `legal-history-hub/` cloned locally
- Google Chrome open and signed in to the Google Sheet
- The **"Claude in Chrome"** extension installed from the Chrome Web Store
- Chrome integration enabled: in the **terminal** with `/chrome`, in the **VS Code extension** via an `@browser` mention in the prompt field

Claude Code drives your Chrome tab

Native integration, no MCP: open Sheet, read cells, click, take screenshots. Visible in a real browser window.

When it stalls ("*extension not detected*"):

- Terminal: `/chrome` → *reconnect extension*
- VS Code: send `@browser` in the prompt again
- If all else fails: restart Claude Code and Chrome

Goal: you experience Claude Code understanding your data model, explaining it, and finding errors that Sheets alone cannot see.

Exercise 1: have the model explained to you

Prompt Claude Code one after another:

1. "*Open our Google Sheet in Chrome and explain the hybrid model in your own words.*"
2. "*What is the difference between the core tab (wide) and the people tab (long)? Use proj-001 as an example.*"
3. "*Why does title_de appear in every row of the long tabs even though it is already in core?*"

Goal: Claude Code **understands and explains** the model, not just executes code.

Exercise 2: add a new field

Suggest a field (e.g. contact person, DOI, project lead).

Then ask Claude Code: "*I want to store [this field] per project. Where should it go?*"

The questions Claude Code should ask:

Question	Answer → consequence
How many contact persons per project?	Exactly one → **wide** in `core`
Could there be more than one?	Yes → **long** in `people` with role `contact`
Does the email need its own slot?	→ new column in `core`, or in `authority`

Learning goal: apply the rule of thumb from Block 1 to a real case.

Exercise 3: have errors found

The exercise Sheet contains **deliberate errors**.

Prompt Claude Code:

- "*Check the tabs against each other. Find inconsistencies.*"
- "*Show me every row in the people tab whose role is not in vocabulary[person_roles].*"
- "*Are there project_ids in the long tabs that don't exist in core?*"
- "*Are there keywords in keywords that are missing from authority?*"

The four hidden errors

#	Error	Type
1	`role = "mitarbeiter"` instead of `researcher`	unknown enum value
2	`project_id = "proj-099"` in people	dangling foreign key
3	keyword `natural-law` not in authority	missing enrichment
4	`proj-006` without `title_de`	empty required field

Claude Code finds them and explains them. You fix them in the Sheet, Claude checks again.

Find errors, fix them, check again. This cycle is the core of the workflow.

Block 2: takeaways

- Claude Code can **read and explain** your data model, not just run code.
- The **wide-vs-long decision** is a repeatable process: "How many values? One or many?"
- **Referential integrity** (do the IDs line up?) is something Claude Code can check, but Sheets alone cannot.
- **The three rules from Block 1 still hold:** verify, work small, a report is not proof.

Break (10–15 min)

After the break: building the Sheet as a CMS.

Block 3: Google Sheets as a CMS

Now we set up our Sheet so that it works well as an **editor**: dropdowns, validation, views.

Goal: the Sheet knows the rules of the data model and helps enforce them.

Step 1: convert tabs to Tables

`Format → Convert to table`

What Tables give you:

- **Column types**: Number rejects text, Date rejects free text
- **Auto-expansion**: new rows inherit formatting, validation, formulas
- **Named references**: the table is called `core`, `people`, etc.
- **Filters in the header**: built in, no extra filter view

Tables are not styling. They are a **data object** with rules.

https://support.google.com/docs/answer/14239833

Structured References: formulas you can read

Classic (fragile):

```
=VLOOKUP(A2, core!A:B, 2,
         FALSE)
```

The `2` means "second column". Breaks on every column insertion.

With Tables (stable):

```
=XLOOKUP(A2,
  core[project_id],
  core[title_de], "")
```

Reads like a sentence. Survives column insertions.

The syntax `core[project_id]` literally means: "*the column* `project_id` *of the table* `core`".

How do you type `core[project_id]`?

Not from memory. Sheets helps.

1. Start the formula: `=XLOOKUP(A2,`
2. Type the table name: `co...` → Sheets suggests `core`
3. Square bracket `[` → Sheets shows all columns of the table as a list
4. Pick a column → Sheets closes the bracket `]` automatically

Prerequisite: the tab must have been converted to a Table via `Format → Convert to table` first. Otherwise Sheets does not know the name.

The locale separator caveat

> **Watch out:** German Sheets uses a **semicolon** as the argument separator (because the comma is the decimal mark, e.g. `3,14`). Other locales follow the same logic.

Online examples and AI output almost always use commas. Paste → `#ERROR!`.

Wrong (English locale)	Right (German locale)
`=XLOOKUP(A2, core[project_id], core[title_de], "")`	`=XLOOKUP(A2; core[project_id]; core[title_de]; "")`

On `#ERROR!`, replace commas with semicolons first.

Step 2: title_de as a reading aid

Add a formula column `title_de` after `project_id` in every long tab:

```
=XLOOKUP(A2; core[project_id]; core[title_de]; "")
```

- New row → enter `project_id` → title appears immediately
- Change the title in `core` → updates everywhere

`core.title_de` stays the **source of truth**, all other occurrences are derived views.

Step 2b: when auto-expansion does not kick in

Google Sheets Tables normally copy formulas into new rows automatically. If they do not:

`Ctrl+D` **(Fill Down)** is the universal fallback.

1. Select the cell with the working formula
2. Extend the selection down to the last row (`B2:Bn`)
3. Press `Ctrl+D`

The formula spreads across all selected cells, with row references adjusted. Works outside Tables too.

Step 3: three layers of validation

Behind every dropdown sits a **source**. Our Sheet has three:

Layer	Tab	Role
vocabulary	`vocabulary`	closed enum lists (roles, status)
authority	`authority`	authority file with PIDs (ORCID, GND, ROR)
_helpers	`_helpers`	filtered views per type, bound to dropdowns

> *vocabulary is the rulebook, authority is the lookup, _helpers is the lens.*

Authority is **never** bound directly to a dropdown, always through `_helpers`. Why: next slide.

Reject input vs. show warning

The data validation panel offers two modes:

Reject input

Red X, value is **not** stored.

For real enums where wrong values must be impossible: `status`, `role`.

Show warning

Red corner, value **is** stored.

For foreign IDs and authority bindings, where new values get backfilled later: `person`, `keyword`, `project_id`.

Rule of thumb: *reject* only on real enums from `vocabulary`. Otherwise always *warn*, so the Sheet can grow without blocking.

Why three layers instead of one?

Problem: `authority` mixes everything – people, institutions, keywords, regions. A dropdown bound directly to it would mix all types together.

Solution: `_helpers` filters:

```
=FILTER(authority[label]; authority[type]="person")
```

`person_labels` shows only people, `keyword_labels` only keywords. Thanks to the **spill range**, the list grows with the source – no manual extension.

Dropdown architecture at a glance

Target cell	Binding	Layer	Mode
`core.status`	`vocabulary[status_values]`	vocabulary	reject
`people.role`	`vocabulary[person_roles]`	vocabulary	reject
`people.person`	`_helpers[person_labels]`	_helpers	warn
`institutions.institution`	`_helpers[institution_labels]`	_helpers	warn
`institutions.relation`	`vocabulary[institution_relations]`	vocabulary	reject
`subjects.subject`	`_helpers[subject_labels]`	_helpers	warn
`keywords.keyword`	`_helpers[keyword_labels]`	_helpers	warn
every `project_id`	`core[project_id]`	foreign key	warn

Warn for everything that may grow. **Reject** only for real enums.

Dropdowns in action

	A	B	C	D	E	F	G	H	I
1	project_id	title_de	person	role					
2	proj-001	Die Schule von Salamanca. Eine digita	Duve, Thomas	PI					
3	proj-001	Die Schule von Salamanca. Eine digita	Lutz-Bachmann, Matthias	PI					
4	proj-001	Die Schule von Salamanca. Eine digita	König, Florian	researcher					
5	proj-001	Die Schule von Salamanca. Eine digita	Birr, Christiane	researcher					
6	proj-001	Die Schule von Salamanca. Eine digita	Rico Carmona, Cindy	researcher					
7	proj-001	Die Schule von Salamanca. Eine digita	Solonets, Polina	researcher					
8	proj-001	Die Schule von Salamanca. Eine digita	Wagner, Andreas	researcher					
9	proj-001	Die Schule von Salamanca. Eine digita	Hugel, Marie-Astrid	researcher					
10	proj-001	Die Schule von Salamanca. Eine digita	Schweighöfer, Stefan	project-coordinator					
11	proj-002	Nichtstaatliches Recht der Wirtschaft.	Collin, Peter	PI					
12	proj-002	Nichtstaatliches Recht der Wirtschaft.	Wolf, Johanna	researcher					
13	proj-002	Nichtstaatliches Recht der Wirtschaft.	Wagner, Andreas	researcher					
14	proj-002	Nichtstaatliches Recht der Wirtschaft.	Solonets, Polina	researcher					
15	proj-002	Nichtstaatliches Recht der Wirtschaft.	Ebbertz, Matthias	researcher					
16	proj-002	Nichtstaatliches Recht der Wirtschaft.	Vesper, Tim-Niklas	researcher					
17	proj-002	Nichtstaatliches Recht der Wirtschaft.	Michel, Lisa	student-assistant					
18	proj-002	Nichtstaatliches Recht der Wirtschaft.	Gödde, Ben	student-assistant					
19	proj-002	Nichtstaatliches Recht der Wirtschaft.	Wolf, Paulina	student-assistant					

Why were the dropdowns hard-coded?

Gemini-in-Sheets built the dropdowns during preparation. You remember the hallucination story from Block 1 (Gemini reports success where there was none). Here a **different Gemini weakness** kicks in: dropdowns *from a range* are structurally not supported. Only **single-value lists**.

Google documents this themselves:

> *"Selecting from a range is not supported in Gemini's dropdown creation feature."*

This is not a bug, it is a **structural limit**. Dynamic problem (hallucination) vs. static problem (feature limit). You need to know both. That is why we now rebuild the dropdowns to use dynamic range bindings, manually.

The 10-second check

How do you tell whether a dropdown is bound correctly?

Inside a Table (after Tables conversion): click the type icon in the column header → *Edit column type*. There you see whether the column type is *Drop-down* or *Drop-down (from a range)*.

You see …	Meaning
"*Drop-down*" (single-value list)	**Hard-coded.** Rebuild!
"*Drop-down (from a range)*" with a reference like `vocabulary[person_roles]`	**Dynamically bound.** Correct.

Outside Tables: *Data* → *Data validation* shows the same. You can apply this check to any Sheet.

Filter views: focus on long tabs

`Data → Filter views → Create new filter view`

Filter on `project_id = "proj-001"` → save as "*proj-001 people*"

What this gives you:

- You see **only the rows** for one project
- You edit **in focus** without disturbing other rows
- Each user can have **their own views** at the same time

Filter view or grouping?

Goal	Better choice
Edit one project (only its rows)	**Filter view**
Overview of all projects with counts	**Grouping view**
Cross-project analysis (all PIs)	**Group by** `role`

Both are **per-user** and don't interfere with each other.

Conditional formatting

	A	B	K	L	M	N	O	P	Q	R	S	T
1	project_id	title_de	status	url1	url2	url3	date_added	verified				
2	proj-001	Die Schule von Salamanca. Eine digitale Quellensa	active	https://www.lhlt.mpg.de/gemeinschaftsprojekt/di	https://www.salamanca.school/		27.02.2026	v				
3	proj-002	Nichtstaatliches Recht der Wirtschaft. Die normat	completed	https://www.lhlt.mpg.de/3455831/jp-collin-non-st	https://metallindustrie.lhlt.mpg.de/		19.03.2026	v				
4	proj-003	Historisches Wörterbuch des kanonischen Rechts	active	https://www.lhlt.mpg.de/research-project/historic	https://dch.hypotheses.org/		19.03.2026	v				
5	proj-004	Gonzalo Fernández de Oviedo	active	https://www.lhlt.mpg.de/3285537/02-damler-gon			19.03.2026	v				
6	proj-005	Verhandelte Ordnung. Arbeitsbeziehungen in der V	completed	https://www.lhlt.mpg.de/2524472/jp-ebbertz-worl			19.03.2026	v				

Setting up conditional formatting

Example: completed projects should colour the whole row grey.

1. Select the data range: click into the table, `Ctrl+A` captures everything (with Tables). Otherwise manually, e.g. `A2:N50`.
2. `Format → Conditional formatting`
3. In the right panel: **"Custom formula is"**
4. Enter the formula: `=$F2="completed"` (column F = `status`)
5. Pick a background colour (grey), save

Why the `$` in `=$F2="completed"`?

- `$F` means: *always* check column F, regardless of which column the cell is in
- `2` without `$` means: adjust the row when the rule iterates over other rows

Result: row 2 turns grey when `F2 = "completed"`. The entire row, not just the status cell.

Without `$F`, the rule in column B would look at column B – there is no `status` there, nothing would turn grey.

From editor to hub: how does the data get out?

Our Sheet is now a proper CMS. But the Hub does not read a Google Sheet directly – it needs a static JSON file.

The bridge: a Python script that reads via the Google Sheets API, joins, validates, and writes `projects.json`. Claude Code built it, Claude Code runs it.

Next slides: what the Sheets API is, and what the pipeline looks like in detail.

What is the Sheets API?

An **API** (Application Programming Interface) is an interface through which programs talk to each other. The **Google Sheets API** lets Claude Code access the Sheet directly, without browser clicks.

Chrome integration (Block 2)	Sheets API (pipeline)
Claude drives the browser	Claude queries the Sheets server directly
Good for: reading, screenshots, single edits	Good for: bulk reading, build pipelines
Slow, visual	Fast, structured
"Claude in Chrome" + `/chrome` resp. `@browser`	credentials prepared in advance

Both have their place. The API is the professional path for the build.

Hands-on: walking through the pipeline once

```
Google Sheets (9 tabs)
    ↓ Claude Code runs scripts/build-hub-data.py
    ↓ script reads via Sheets API, joins, validates
data/projects.json (nested, enriched)
    ↓ git add, commit, push
GitHub Pages (Hub live)
```

1. Add a new person in Sheets
2. Claude Code: "*Rebuild* `projects.json` " → runs the script
3. Inspect, commit, push the result
4. Hub shows the new person

The script is ready. We will look inside it in Block 4.

Block 3: takeaways

- **Tables** are data objects, not styling. Structured References make formulas readable.
- **Three layers**: vocabulary (rulebook), authority (lookup), _helpers (lens).
- **Reject vs. warn:** reject only for enums, warn everywhere else.
- **Filter views** make long tabs usable day-to-day.
- **The pipeline**: Sheets → Claude Code (API) → `projects.json` → Hub.

Block 4: Skills and plugins

Claude Code can do more than single prompts. **Skills** and **plugins** make recurring tasks repeatable.

What are Skills?

- Reusable commands that Claude Code triggers with `/`
- **Prompt wrappers**: one word activates a structured instruction set, which Claude executes with **judgement** – not as a macro, but situationally
- A Skill is a **folder** containing a `SKILL.md` file (and possibly helpers)

Reference: https://code.claude.com/docs/en/skills.md

Skills: two locations

```
legal-history-hub/
  .claude/skills/           ← project-local, versioned in the repo
    validate-data/
      SKILL.md
    enrich-authority/
      SKILL.md

~/.claude/skills/           ← global, across all projects
  promptotyping/
    SKILL.md
```

Local for Hub-specific things, global for anything you need across multiple projects.

Installing Skills: `npx skills`

Skills live as folders in GitHub repos. The Vercel Labs CLI pulls them onto your machine:

```
npx skills add anthropics/skills --skill skill-creator -g -a claude-code
npx skills add DigitalHumanitiesCraft/promptotyping-skill -g -a claude-code
```

- `-g` = global (`~/.claude/skills/`), without `-g` project-local
- `-a claude-code` = target agent (also `cursor` , `codex` , ...)
- `npx skills list` / `update` for overview and updates

Script or Skill?

Not every problem is a Skill problem.

	Python script	Skill (`SKILL.md`)
Input → output	deterministic, same on same input	variable, context-dependent
Failure mode	exception, stack trace	hallucination possible
Automatically testable (CI*)	yes	no
Adjustment	edit code, test	edit prompt
Hub example	`build-hub-data.py` (Sheet → JSON)	`/explain-model`, `/enrich-authority`

Rule: *same input → same result = script. Claude's judgement is part of the value = Skill.*

*CI = Continuous Integration. Automated test runs on `git push`. Scripts run there, Skills do not.

Live demo: /promptotyping

We invoke `/promptotyping` and watch.

The Skill reads the project's promptotyping documents and suggests which document type would make sense next (RESEARCH, REQUIREMENTS, DESIGN, JOURNAL).

Then we open the Skill file in the editor:

"*This is a Markdown document with instructions. Claude Code reads it and follows the steps.*"

So you can write your own Skills. Anything you can phrase in natural language can become a Skill.

Prerequisite: /skill-creator

You don't simply write a Skill into a Markdown file. Anthropic ships a **meta-Skill** for this: `/skill-creator`. It knows what a good Skill looks like.

It takes care of:

- clean frontmatter (`name`, `description`)
- a precise `description` – this is how Claude knows when the Skill should fire
- progressive disclosure (details in helper files, not everything in `SKILL.md`)

Without `/skill-creator`, Claude writes a Markdown file that *looks* like a Skill. With `/skill-creator`, it becomes one.

Together: building /validate-data live

We build a **Skill**, not a script. The difference:

- A script lists errors raw to `stdout`
- A Skill **explains** findings in natural language, **prioritises**, **proposes fixes**

The Skill advantage: natural-language explanation for editor work, not just error codes. A pure checker would be better as a Python script.

/validate-data: the flow

1. Invoke `/skill-creator`. On the prompt, describe: "*A Skill `/validate-data` that reads the Sheet via the Chrome integration, checks all `project_id` in the long tabs against `core`, `role` and `relation` against `vocabulary`, and explains in clear sentences what is wrong and how to fix it.*"
2. Claude Code writes `.claude/skills/validate-data/SKILL.md`
3. We read the file together
4. We invoke `/validate-data` – Claude Code picks up new Skills live, no restart
5. Refine: "*Show me the concrete row, the project, and a fix suggestion for each finding.*"

Skills and scripts for the Hub

Type	Name	What it does	Status
Script	`build-hub-data.py`	Sheets → `projects.json` (join + enrichment)	exists
Skill	`/explain-model`	explains the model state in clear sentences	idea
Skill	`/add-project`	guided dialogue for new projects	idea
Skill	`/validate-data`	findings + fix suggestions	next
Skill	`/enrich-authority`	ORCID/GND gaps with approval loop	idea

Brainstorm: which Skill would you need next?

MCP plugins: new capabilities

Skills tell Claude Code **what** to do. **MCP plugins** give Claude Code new **tools**.

MCP = **Model Context Protocol**, a standard for the wire between AI and external services.

Examples: **Google Sheets MCP** (API access to the Sheet), **ORCID API plugin** (researcher IDs), **GitHub MCP** (issues/PRs).

The Chrome integration from Block 2 is **not** an MCP plugin, it's built in natively. You don't have to build plugins yourselves – the important thing is to know they exist.

CLAUDE.md: the project memory

On every start, Claude Code reads `CLAUDE.md` in the project root – the manual for this project.

It contains:

- which tabs the Sheet has
- how the hybrid model is built
- which conventions apply
- which Skills are available

When Claude Code "knows" your model, it is because of this file. If something is missing or stale: update `CLAUDE.md`.

Block 4: takeaways

- **Skills** = prompt wrappers for workflows that need Claude's judgement. `.md` files in `.claude/skills/`.
- **Scripts** = deterministic automation (Python, shell). Claude Code writes and maintains them, but they are **not** Skills.
- **Rule of thumb:** same input, same output → script. Judgement needed → Skill.
- **MCP plugins** = new tools (APIs, Sheet access). They exist, you don't have to build them.
- **CLAUDE.md** = project memory. Claude Code "knows" only what is written there.

Block 5: wrap-up

Learnings

- Describe and justify a data model as a **hybrid of wide + long**
- Apply the rule of thumb: singleton → wide, many-to-many → long
- Set up dropdowns that bind to `vocabulary` or `_helpers`
- Create filter views and grouping views per project
- Use Claude Code to **read, explain, extend, validate** a model
- Run the flow **Sheets** → **Claude Code (API)** → `projects.json` → **Hub** yourselves
- Recognise Skills as reusable instructions and **formulate your own ideas**
- **Verify AI results** instead of trusting self-reports

The bigger picture

Sheets is the editor, not the model.
The power (joins, enrichment, PIDs) emerges at build time in Claude Code.

The data model is not a one-shot.
Pilot experiment → hybrid model: exactly that kind of conversation. Intermediate steps are not failures.

Controlled vocabulary + authority lookup is the one place where normalisation makes sense. Everything else is allowed to be flat and readable.

Think validation! Check every AI result against the artefact – Sheet content, git diff, file state. Self-reports are claims, not proofs.

Outlook: WS4 – WS6

Workshop	Topic
WS4	Claude infrastructure in everyday work. Context engineering, Claude Desktop, Projects, Artifacts, Memory, the product landscape (Excel, Word, PowerPoint, Chrome, Code). Which tool when.
WS5	Open – topics emerge from WS4 and your needs.
WS6	Open – polish, integration with collaboration processes, or a use case of your own.

In every workshop: always with Claude Code, always iterative.

Thank you!

Resources:

- Tutorial Lesson 3: *Understanding the data model*
- Cheat sheet: rules of thumb, three-layer dropdowns, top-10 prompts
- `CLAUDE.md` and `README.md` in the repo

Questions? Ideas? Feedback?